

Javascript

Theory

1. Promises

Definition: A Promise is an object representing the eventual completion or failure of an asynchronous operation.

States:

- **Pending** → initial state.
- **Fulfilled** → operation completed successfully.
- **Rejected** → operation failed.

Syntax:

```
const promise = new Promise((resolve, reject) => {  
  
  let success = true;  
  
  if(success) resolve("Done");  
  
  else reject("Error");  
  
});
```

2. Promise Chaining

Definition: Using `.then()` sequentially to handle results step by step.

Use case: Cleaner handling of asynchronous tasks compared to nested callbacks.

Syntax:

```
fetchData()  
  
  .then(res => res * 2)  
  
  .then(res => res * 3)  
  
  .catch(err => console.log(err));
```

3. Callbacks

Definition: A callback is a function passed as an argument to another function, executed later.

Problem: Can lead to **callback hell** when nested deeply.

Syntax:

```
function greet(name, cb){  
  
    console.log("Hello", name);  
  
    cb();  
  
}
```

4. Classes and Static Methods

Definition: ES6 classes provide a blueprint for objects.

Static methods: Belong to the class, not instances.

Syntax:

```
class Bank {  
  
    static greet(name){ console.log("Hello", name); }  
  
}  
  
Bank.greet("FIIT");
```

5. Async vs Sync

- **Synchronous** → executes line by line.
- **Asynchronous** → executes tasks in background (e.g., Promises, callbacks, async/await).

Coding Round Questions

- Promise basics → Explain what a Promise is and its states.
- Promise chaining → Write a program to multiply values step by step using Promise chaining.
- Callback → Show how a callback function works with a greeting example.
- Static method → Create a class with a static method and call it.
- Promise rejection → Demonstrate how to handle rejected Promises using .catch().

⚡ Machine Coding Round Challenge

Problem: Order Processing System

Requirements:

- Create a function placeOrder() that returns a Promise.
- If stockAvailable = true, resolve with "Order placed successfully".
- If not, reject with "Out of stock".
- Chain .then() calls to:
 - Confirm order.
 - Process payment.
 - Ship order.
- Use .catch() to handle errors.
- Add a Customer class with a static method welcome().

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Promises</title>
  </head>
  <body>
    <h1>Promise</h1>
    <script>
      function fetchData() {
        return new Promise((resolve) => {
          setTimeout(() => {
            resolve(1);
```

```

    }, 2000);
  });
}

fetchDatta()
  .then((result) => {
    console.log(result); //1
    return result * 2;
  })
  .then((result) => {
    console.log(result); // 2
    return result * 3;
  })
  .then((result) => {
    console.log(result); // 6
  });

//   setTimeout(() => {
//     console.log("timeout");
//   }, 5000);

//   const id = setInterval(() => {
//     console.log("Interval");
//   }, 2000);

//   setTimeout(() => {
//     clearInterval(id);
//   }, 15000);

console.log("hello");

let classExpression = class Bank {
  static greet(name) {
    console.log("hello there", name);
  }
};

let obj = new classExpression();

//   obj.greet(); // belongs to object

classExpression.greet("FIIT");

// callback

function sayHello(name, cb) {
  console.log("Hello ", name);
  cb(secondCallback);
}

```

```
}

function nameCallback(cb) {
  console.log("Name callback is executed");
  cb();
}

function secondCallback() {
  console.log("second callback is executed successfully");
}

sayHello("Arun", nameCallback);

const promise = new Promise((resolve, reject) => {
  let success = false;

  if (success) {
    resolve("Promise is fulfilled");
  } else {
    reject("promise is rejected");
  }
});

promise
  .then((result) => console.log(result))
  .catch((result) => console.log(result));
</script>
</body>
</html>
```